

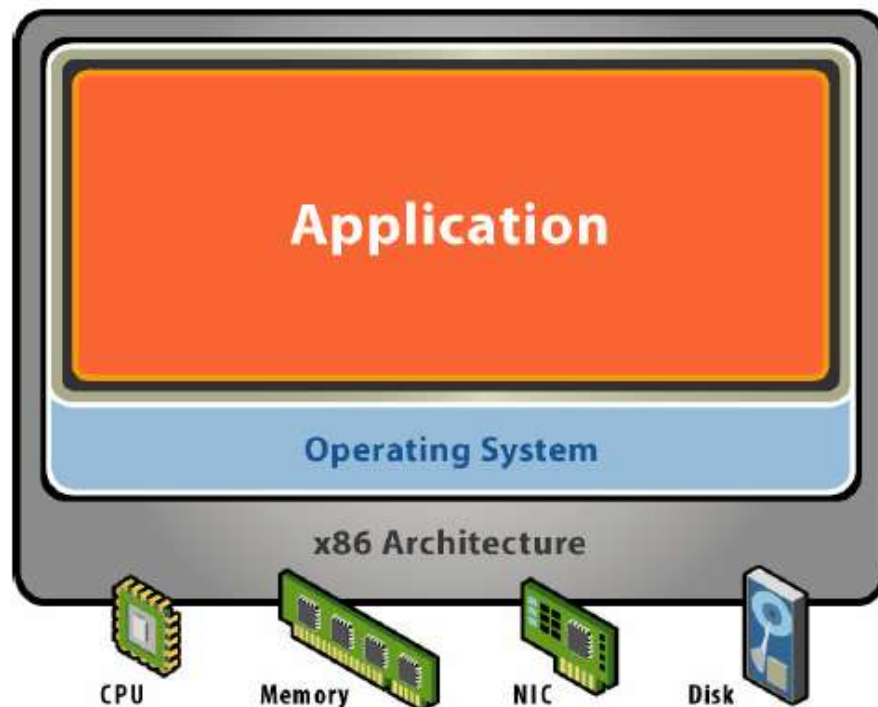
Virtual Machines

Virtualization

- Virtualization deals with “extending (or abstracting) an existing interface so as to mimic the behavior of another system”
- Virtual system examples: virtual private network, virtual memory, virtual machine



Starting Point: A Physical Machine



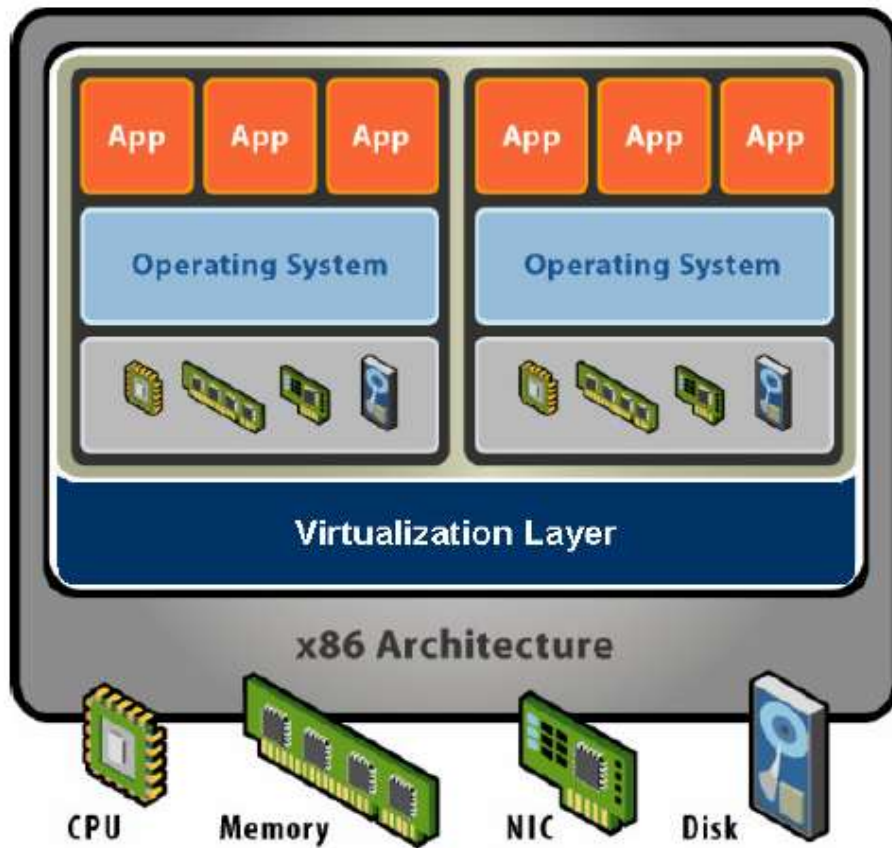
Physical Hardware

- Processors, memory, chipset, I/O bus and devices, etc.
- Physical resources often underutilized

Software

- Tightly coupled to hardware
- Single active OS image
- OS controls hardware

What is a Virtual Machine?



Hardware-Level Abstraction

- Virtual hardware: processors, memory, chipset, I/O devices, etc.
- Encapsulates all OS and application state

Virtualization Software

- Extra level of indirection decouples hardware and OS
- Multiplexes physical hardware across multiple "guest" VMs
- Strong isolation between VMs
- Manages physical resources, improves utilization

VM Isolation



Secure Multiplexing

- Run multiple VMs on single physical host
- Processor hardware isolates VMs, e.g. MMU

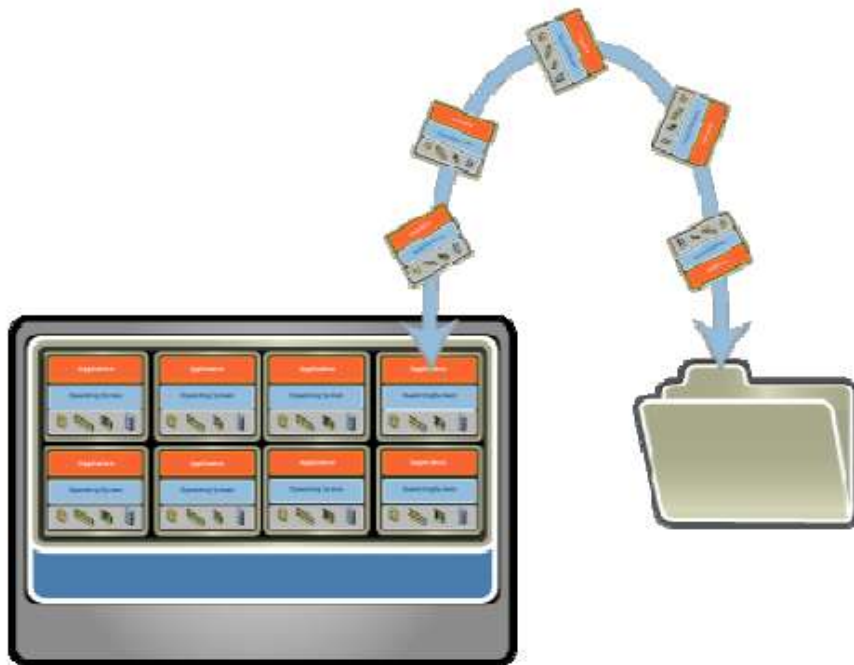
Strong Guarantees

- Software bugs, crashes, viruses within one VM cannot affect other VMs

Performance Isolation

- Partition system resources
- Example: VMware controls for reservation, limit, shares

VM Encapsulation



Entire VM is a File

- OS, applications, data
- Memory and device state

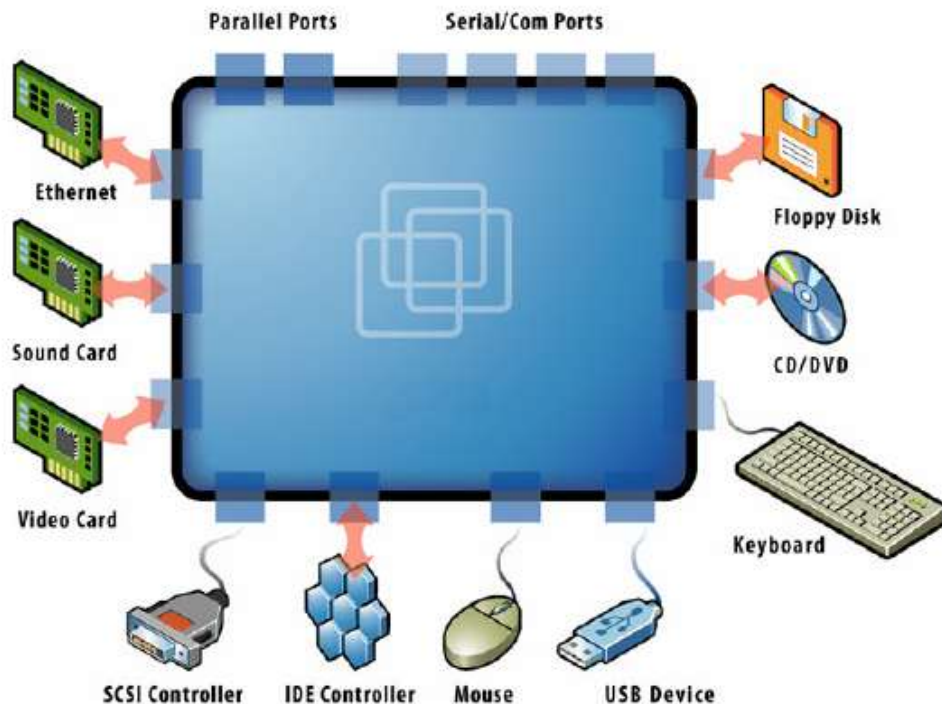
Snapshots and Clones

- Capture VM state on the fly and restore to point-in-time
- Rapid system provisioning, backup, remote mirroring

Easy Content Distribution

- Pre-configured apps, demos
- Virtual appliances

VM Compatibility



Hardware-Independent

- Physical hardware hidden by virtualization layer
- Standard virtual hardware exposed to VM

Create Once, Run Anywhere

- No configuration issues
- Migrate VMs between hosts

Legacy VMs

- Run ancient OS on new platform
- *E.g.* DOS VM drives virtual IDE and vLance devices, mapped to modern SAN and GigE hardware

Common Virtualization Uses Today



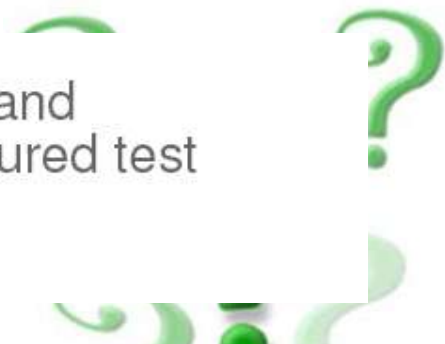
Test and Development – Rapidly provision test and development servers; store libraries of pre-configured test machines



Business Continuity – Reduce cost and complexity by encapsulating entire systems into single files that can be replicated and restored onto any target server



Enterprise Desktop – Secure unmanaged PCs without compromising end-user autonomy by layering a security policy in software around desktop virtual machines

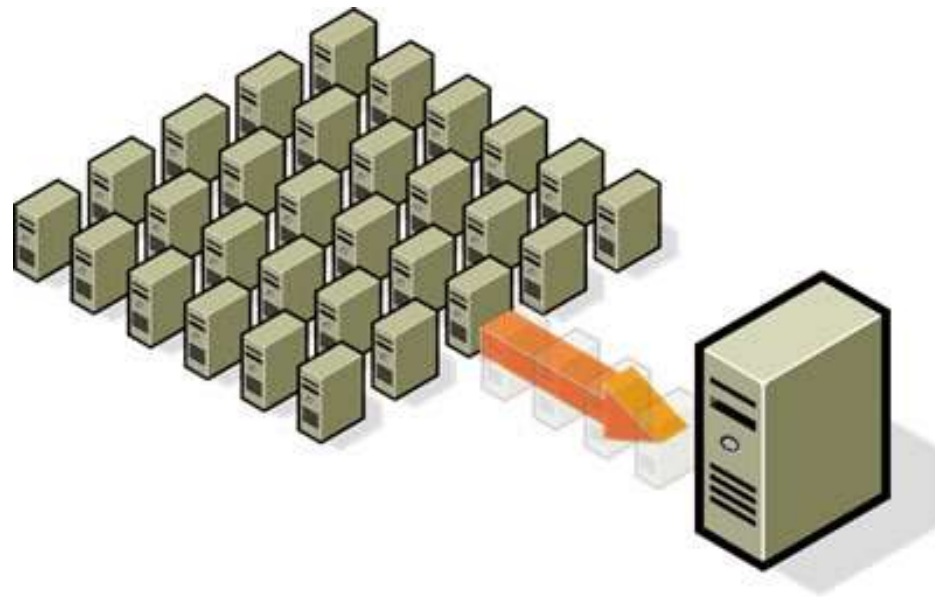


Common Virtualization Uses Today

- Run legacy software on non-legacy hardware
- Run multiple operating systems on the same hardware
- Create a manageable upgrade path
- Manage outages (expected and unexpected) dynamically

Common Virtualization Uses Today

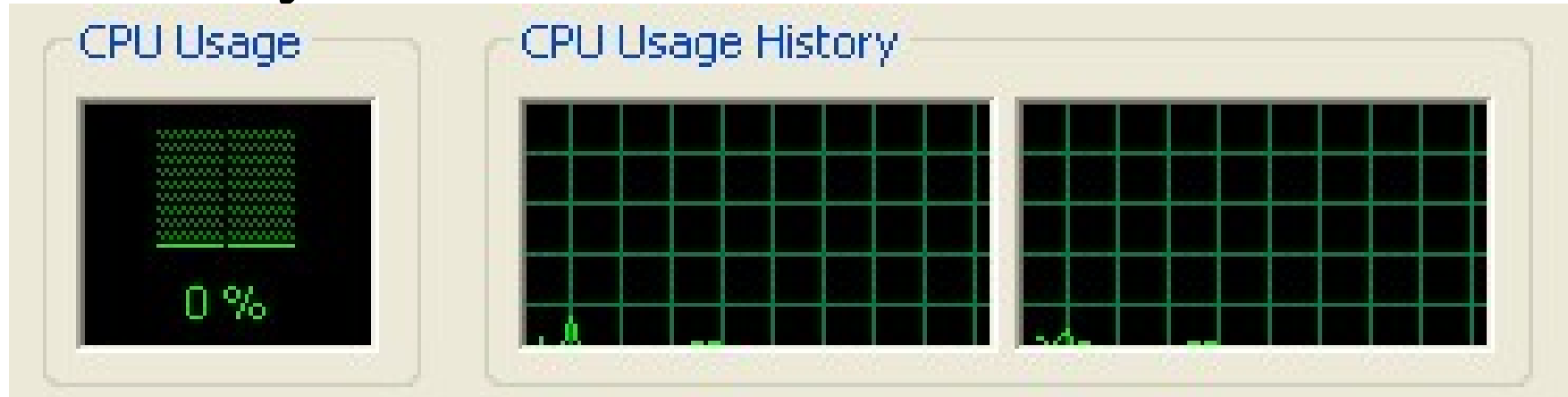
- Reduce costs by consolidating services onto the fewest number of physical machines



<http://www.vmware.com/img/serverconsolidation.jpg>

Non-virtualized Data Centers

- Too many servers for too little work



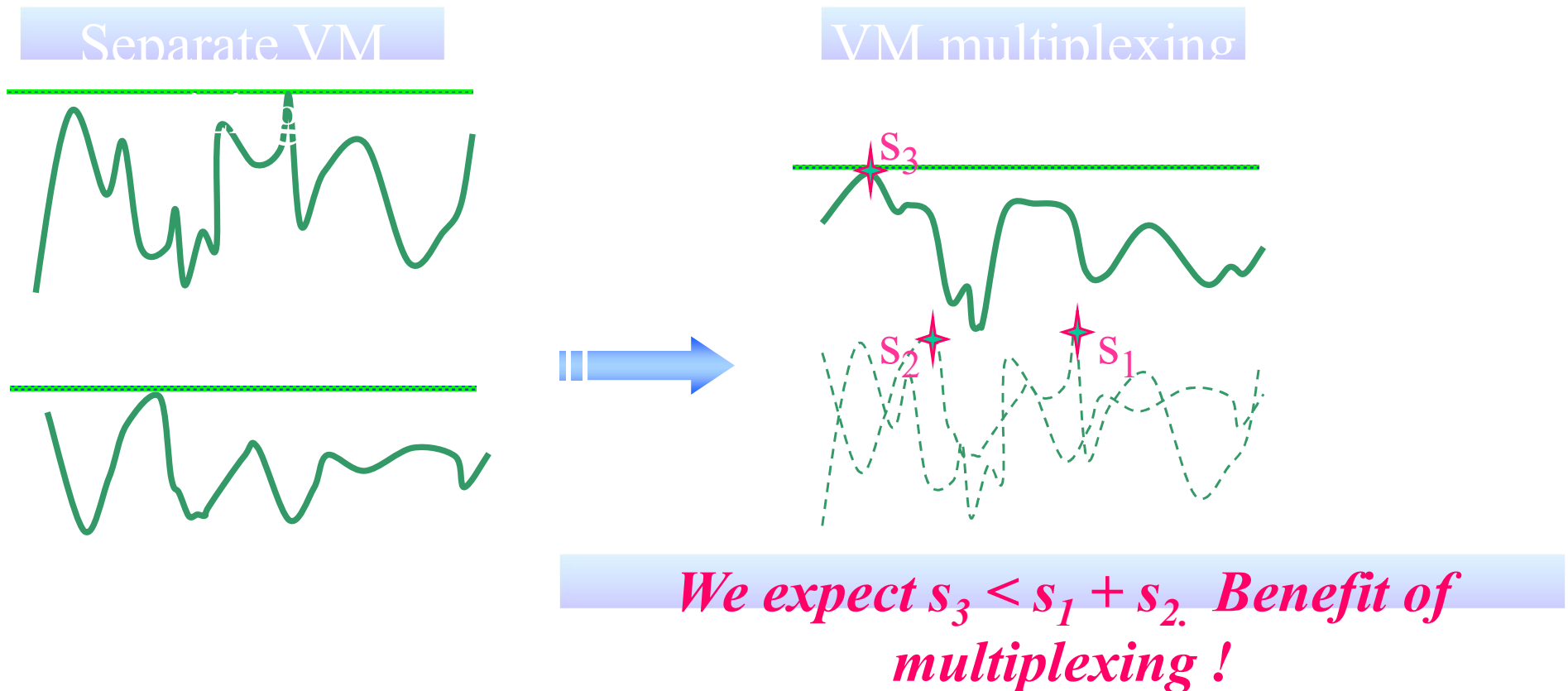
- High costs and infrastructure needs
 - Maintenance
 - Networking
 - Floor space
 - Cooling
 - Power
 - Disaster Recovery



Dynamic Data Center

- Virtualization helps us break the “one service per server” model
- Consolidate many services into a fewer number of machines when workload is low, reducing costs
- Conversely, as demand for a particular service increases, we can shift more virtual machines to run that service
- We can build a data center with fewer total resources, since resources are used as needed instead of being dedicated to single services

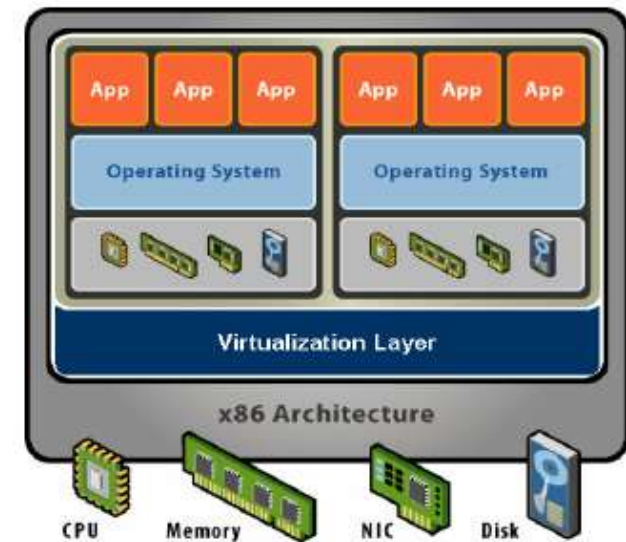
VM workload multiplexing



- Multiplex VMs' workload on same physical server
 - Aggregate multiple workload. Estimate total capacity need based on aggregated workload
 - Performance level of each VM be preserved

What is a Virtual Machine Monitor?

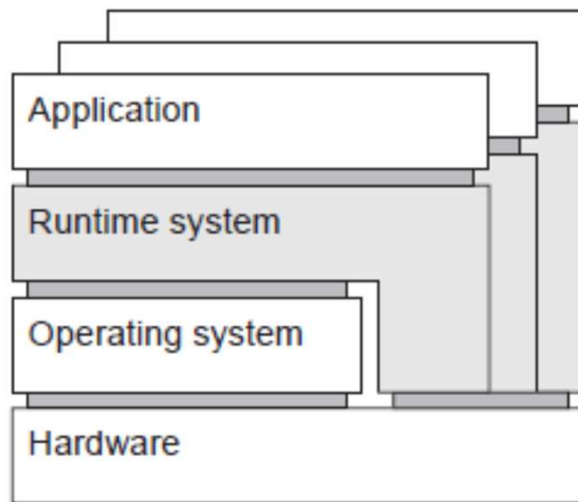
A virtual machine is taken to be an *efficient, isolated duplicate* of the real machine. We explain these notions through the idea of a *virtual machine monitor* (VMM). See Figure 1. As a piece of software a VMM has three essential characteristics. First, the VMM provides an environment for programs which is essentially identical with the original machine; second, programs run in this environment show at worst only minor decreases in speed; and last, the VMM is in complete control of system resources.



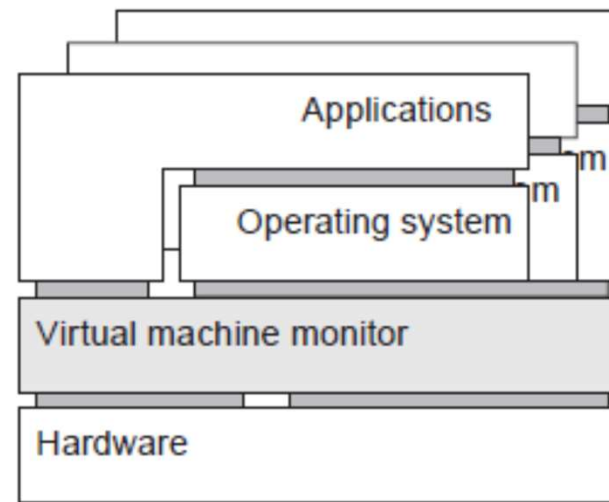
So, it is just like Java VM, right?

- No, a Java VM is very different from the physical machine that runs it
- A hardware-level VM reflects underlying processor architecture

Process VMs versus VM Monitors



(a)



(b)

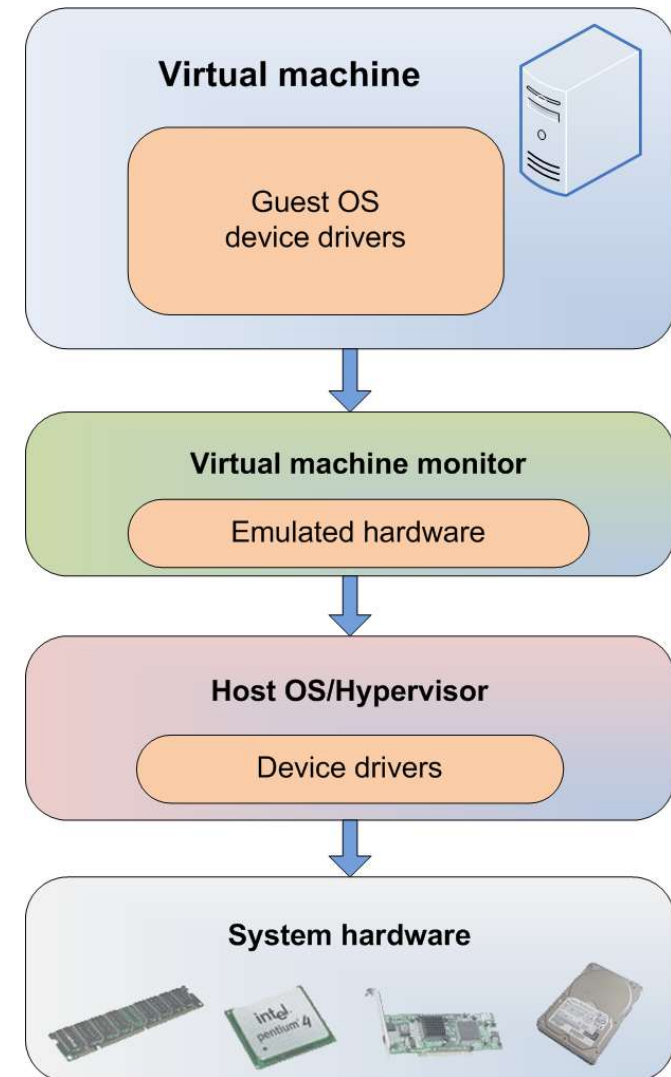
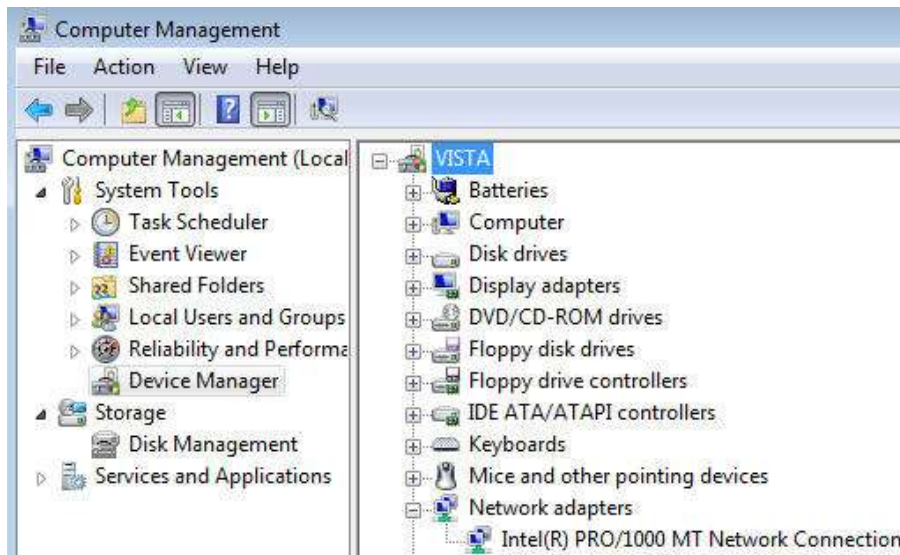
- **Process VM:** A program is compiled to intermediate (portable) code, which is then executed by a runtime system (**Example:** Java VM).
- **VM Monitor:** A separate software layer mimics the instruction set of hardware $\Rightarrow$ a complete operating system and its applications can be supported (**Example:** VMware, VirtualBox).

Thee Virtualization Approaches

- Full Virtualization
- Paravirtualization
- Hardware-assisted Virtualization

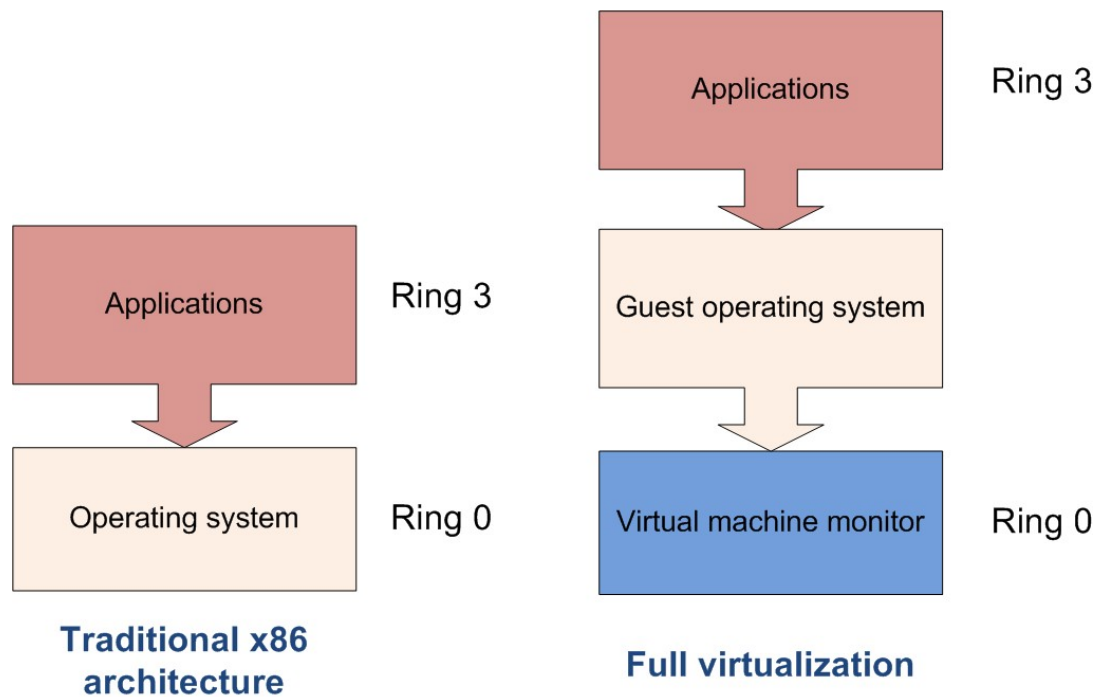
Full Virtualization

- Everything is virtualized
- Full hardware emulation
- Emulation = latency



Privileged Instructions

- Privileged instructions: OS kernel and device driver access to system hardware
- Trapped and emulated by VMM



Pros and Cons – Full Virtualization

- **Pros**
 - Disaster recovery, failover
 - Virtual appliance deployment
 - Legacy code on non-legacy hardware
- **Cons – LATENCY of core four resources**
 - RAM performance reduced 25% to 75%
 - Disk I/O degraded from 5% to 20%
 - Network performance decreased up to 10%
 - CPU privileged instruction dings nearing 1% to 7%

Paravirtualization

—OS or system devices are virtualization aware

Requirements:

—OS level – recompiled kernel

—Device level – paravirtualized or “enlightened” device drivers

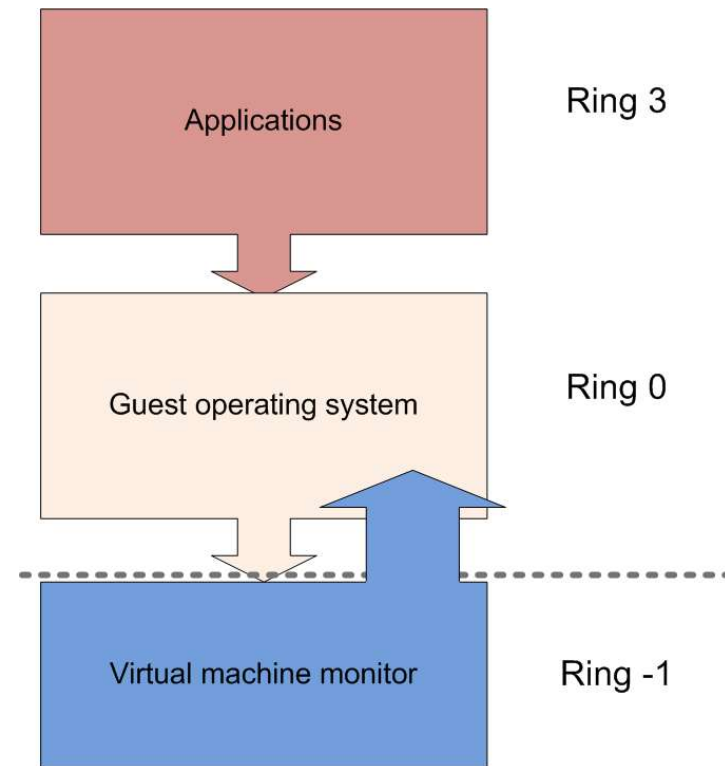


Paravirtualization

- **Pro:** fast
- **Con:** requires a specially modified guest OS, thus precludes the ability to run off-the-shelf and legacy OS in paravirtual environments

Hardware-assisted Virtualization

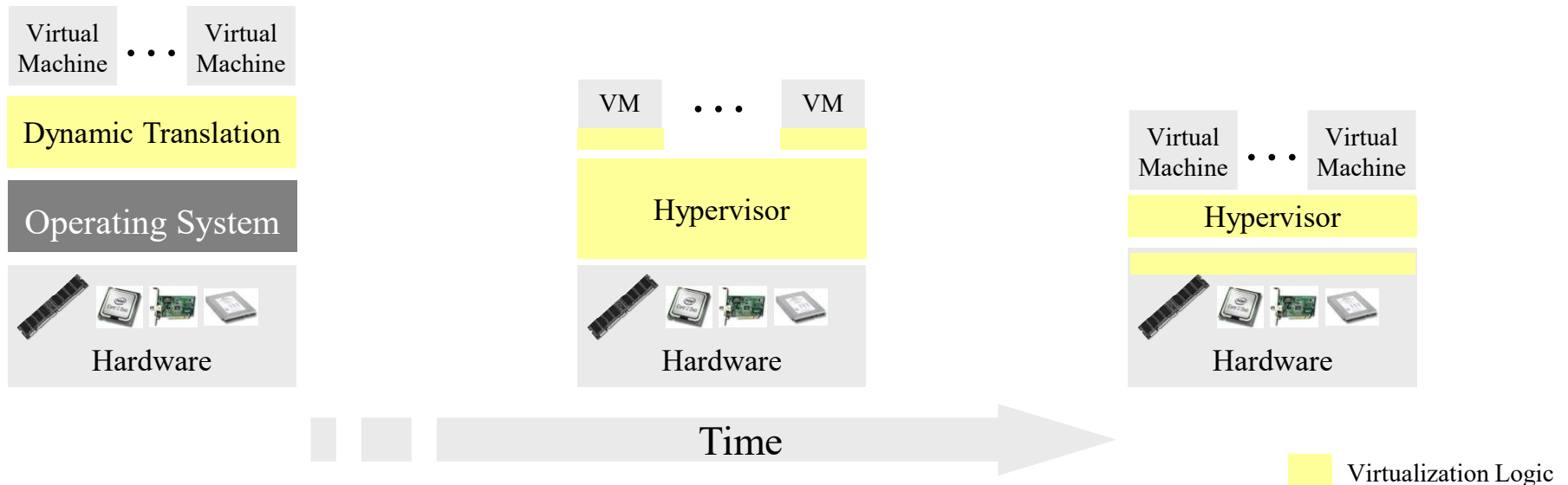
- Server hardware is virtualization aware
- Hypervisor and VMM load at privilege Ring -1 (firmware)
- Removes CPU emulation bottleneck
- Memory virtualization coming in quad core AMD and Intel CPUs



Hardware-assisted virtualization

Evolution of Software solutions*

- 1st Generation: Full virtualization (Binary rewriting)
 - Software Based
 - VMware and Microsoft
- 2nd Generation: Paravirtualization
 - Cooperative virtualization
 - Modified guest
 - VMware, Xen
- 3rd Generation: Silicon-based (Hardware-assisted) virtualization
 - Unmodified guest
 - VMware and Xen on virtualization-aware hardware platforms



*This slide is from Intel® Corporation